33 Setup

33.1 Scenario

Reversible lane direction is scheduled to change while vehicles remain inside.

33.2 Entities

1. Vehicles travelling through the reversible lane.
2. Lane control system at both entrances.
3. Roadside occupancy detectors and entry barriers.
4. Traffic management operator.

33.3 Type of Communication

Inter Vehicle and Infrastructure Communication; Inter Infrastructure Communication.

33.4 Description

A reversible lane is scheduled to change its permitted direction. Entry from the current direction has been closed, but an exit detector still reports a vehicle inside the controlled segment. The controller at the opposite entrance requests permission to open. The two entrances must agree on one lane state and establish that the segment is empty before the direction changes.

33.5 Dialogues

1. Is there a conflict?
Yes. The opposite entrance requests access while a vehicle still occupies the lane in its current direction.

2. Is the reason good?
The request follows the operating schedule, but the schedule alone does not justify admitting opposing traffic.

3. Is settlement good?
Yes, if both entrances remain closed to new entries until clearance is verified. If clearance is uncertain, the change is postponed and an operator is informed.

4. Dialogue type
Deliberation: the controllers must agree on a coordinated sequence using occupancy evidence. Information seeking is needed when detectors disagree.

33 Setup (continued)

33.6 Solutions

1. Close admission at both entrances and record acknowledgements from both controllers.
2. Allow vehicles already inside to leave in their existing direction; monitor the segment and its exits.
3. Compare occupancy readings with exit counts. Treat a missing report or disagreement as an unresolved clearance condition.
4. Once clearance is verified, update direction indicators and interlocks together. Open only the entrance corresponding to the new direction after the other entrance confirms closure.
5. If a vehicle remains inside or either controller fails to acknowledge, keep the change pending and request operator review.

33.7 PEAS

1. Performance Measure - No simultaneous admission of opposing traffic; time between confirmed clearance and reopening; number of unresolved occupancy disagreements.

2. Environment - A controlled reversible road segment with entrances at both ends, direction signs and vehicles already inside.

3. Actuators - Entry barriers, lane direction indicators, vehicle warning displays and operator alert transmitter.

4. Sensors - Lane occupancy detectors, entry and exit counters, barrier position sensors and controller acknowledgement receiver.

33.8 Cascading Problems

1. A detector fails while a vehicle is inside, so the controller cannot prove that the lane is clear.
2. One entrance receives the direction update late and must remain closed until its state is reconciled.
3. Waiting vehicles queue onto a nearby intersection and require separate queue management.

Scenario 33 - Reviewed Class Diagram

AutonomousSystem

- trafficSignals: List<TrafficSignal>
- vehicles: List<Vehicle>
- roadNetwork: RoadNetwork
- scenarioController: ReversibleLaneController

+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignal(trafficSignal: TrafficSignal): void
+ addVehicle(vehicle: Vehicle): void
+ removeVehicle(vehicle: Vehicle): void
+ setRoadNetwork(roadNetwork: RoadNetwork): void
+ runSimulation(): void

RoadNetwork

- intersections: List<Intersection>
- roads: List<Road>

+ addIntersection(intersection: Intersection): void
+ removeIntersection(intersection: Intersection): void
+ addRoad(road: Road): void
+ removeRoad(road: Road): void
+ getIntersections(): List<Intersection>
+ getRoads(): List<Road>

Vehicle

- id: Integer
- topLeft: Point
- bottomRight: Point
- destination: Point
- currentSpeed: Double
- maxSpeed: Double
- sensor: List<Sensor>
- actuator: List<Actuator>

+ move(): void
+ accelerate(): void
+ decelerate(): void
+ changeDirection(): void
+ setDestination(destination: Point): void

TrafficSignal

- id: Integer
- location: Point
- status: SignalStatus
- sensor: List<Sensor>
- actuator: List<Actuator>

+ getId(): Integer
+ getLocation(): Point
+ setStatus(status: SignalStatus): void
+ getStatus(): SignalStatus

0..*

0..*

0..*

0..*

Road

- id: Integer
- start: Intersection
- end: Intersection
- lanes: Integer
- capacity: Integer

+ getId(): Integer
+ getStart(): Intersection
+ getEnd(): Intersection
+ getLanes(): Integer
+ getCapacity(): Integer

start

end

Intersection

- id: Integer
- location: Point
- trafficSignals: List<TrafficSignal>

+ getId(): Integer
+ getLocation(): Point
+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignals(trafficSignal: TrafficSignal): void
+ getTrafficSignals(): List<TrafficSignal>

uses

Point

- x: Integer
- y: Integer

+ getX(): Integer
+ getY(): Integer
+ setX(Integer): void
+ setY(Integer): void

SignalStatus (enum)

RED
YELLOW
GREEN

coordinates vehicles

uses Road

scenario extension

ReversibleLaneController

- laneId: String
- direction: Direction
- clearanceVerified: Boolean
- road: Road
- sensor: List<Sensor>
- actuator: List<Actuator>

+ requestDirectionChange(): void
+ verifyClearance(): Boolean
+ commitDirection(): void

0..*

DirectionChange

- requestedDirection: Direction
- leftClosed: Boolean
- rightClosed: Boolean

+ readyToCommit(): Boolean

reads

0..*

0..*

<<interface>> Sensor

+ sense(): void

0..*

<<interface>> Actuator

+ actuate(): void

commands

LaneOccupancySensor

- occupied: Boolean

+ sense(): void
+ countVehicles(): Integer

BarrierPositionSensor

- closed: Boolean

+ sense(): void
+ confirmClosed(): Boolean

EntryBarrier

- closed: Boolean

+ actuate(): void
+ setClosed(value: Boolean): void

LaneDirectionIndicator

- direction: Direction

+ actuate(): void
+ displayDirection(): void

Reused class signatures follow the original image set. New controllers, records and concrete devices extend that core.

Solid line: association | Filled diamond: composition | Dashed line + hollow triangle: interface realization

The new scenarioController association is additive. Repeated primitive fields express the same types as the originals.

Direction = CLOSED, FORWARD, REVERSE.

34 Setup

34.1 Scenario

Heavy vehicles request bridge entry when their combined load would exceed the permitted operating limit.

34.2 Entities

1. Heavy vehicles approaching the bridge.

2. Bridge admission controller.

3. Weight measurement stations and bridge occupancy detectors.

4. Bridge operator.

34.3 Type of Communication

Vehicle-to-Bridge Infrastructure Communication; Inter Infrastructure Communication.

34.4 Description

Two heavy vehicles arrive at separate bridge entrances. Each vehicle is eligible to use the bridge individually, but their combined load, together with vehicles already on the span, exceeds the operator-configured admission limit. Neither approach has a complete view of the other approach. A shared controller must decide the admission order from verified measurements and current occupancy.

34.5 Dialogues

1. Is there a conflict?
Yes. Both vehicles request immediate entry, but the bridge cannot admit them together under the configured load limit.

2. Is the reason good?
Both requests are valid, but immediate access depends on available load capacity and verified vehicle measurements.

3. Is settlement good?
Yes, if the vehicles accept sequential admission using the stated queue policy. A disputed weight measurement requires rechecking before a permit is issued.

4. Dialogue type
Negotiation over admission order, constrained by a fixed operating limit. Information seeking establishes vehicle weights and whether earlier vehicles have exited.

34 Setup (continued)

34.6 Solutions

1. Read each vehicle's measured weight and check that the measurement is associated with the correct vehicle.
2. Maintain a shared record of vehicles admitted to the span and subtract their load only after their exits are confirmed.
3. Calculate whether the requested entry fits the configured operating limit; reserve capacity before issuing a single-use admission permit.
4. Admit eligible vehicles according to the published queue policy and explain any delay using current occupancy and remaining capacity.
5. If weight or occupancy evidence is uncertain, hold new admissions and ask the operator to resolve the discrepancy.

34.7 PEAS

1. Performance Measure - Admissions remain within the configured load limit; waiting time among eligible vehicles; accuracy of entry and exit accounting.

2. Environment - A load-controlled bridge with two approach queues, measurement stations and an operator-configured admission policy.

3. Actuators - Entry gates, permission displays, queue guidance signs and operator alert transmitter.

4. Sensors - Vehicle weight sensors, vehicle identity readers, span occupancy detectors and exit detectors.

34.8 Cascading Problems

1. An exit is missed, leaving capacity incorrectly reserved and delaying later vehicles.
2. A vehicle enters without using its assigned permit, requiring occupancy reconciliation.
3. A vehicle cannot use an alternative route because that route has a different weight restriction.

Scenario 34 - Reviewed Class Diagram

AutonomousSystem
- trafficSignals: List<TrafficSignal>
- vehicles: List<Vehicle>
- roadNetwork: RoadNetwork
- scenarioController: BridgeAdmissionController

+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignal(trafficSignal: TrafficSignal): void
+ addVehicle(vehicle: Vehicle): void
+ removeVehicle(vehicle: Vehicle): void
+ setRoadNetwork(roadNetwork: RoadNetwork): void
+ runSimulation(): void

RoadNetwork
- intersections: List<Intersection>
- roads: List<Road>

+ addIntersection(intersection: Intersection): void
+ removeIntersection(intersection: Intersection): void
+ addRoad(road: Road): void
+ removeRoad(road: Road): void
+ getIntersections(): List<Intersection>
+ getRoads(): List<Road>

Vehicle
- id: Integer
- topLeft: Point
- bottomRight: Point
- destination: Point
- currentSpeed: Double
- maxSpeed: Double
- sensor: List<Sensor>
- actuator: List<Actuator>

+ move(): void
+ accelerate(): void
+ decelerate(): void
+ changeDirection(): void
+ setDestination(destination: Point): void

TrafficSignal
- id: Integer
- location: Point
- status: SignalStatus
- sensor: List<Sensor>
- actuator: List<Actuator>

+ getId(): Integer
+ getLocation(): Point
+ setStatus(status: SignalStatus): void
+ getStatus(): SignalStatus

0..*

0..*

0..*

0..*

Road
- id: Integer
- start: Intersection
- end: Intersection
- lanes: Integer
- capacity: Integer

+ getId(): Integer
+ getStart(): Intersection
+ getEnd(): Intersection
+ getLanes(): Integer
+ getCapacity(): Integer

start

end

Intersection
- id: Integer
- location: Point
- trafficSignals: List<TrafficSignal>

+ getId(): Integer
+ getLocation(): Point
+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignals(trafficSignal: TrafficSignal): void
+ getTrafficSignals(): List<TrafficSignal>

uses

Point
- x: Integer
- y: Integer

+ getX(): Integer
+ getY(): Integer
+ setX(Integer): void
+ setY(Integer): void

SignalStatus (enum)
RED
YELLOW
GREEN

coordinates vehicles

uses Road

scenario extension

BridgeAdmissionController
- loadLimit: Double
- reservedLoad: Double
- activePermits: List<LoadPermit>
- road: Road
- sensor: List<Sensor>
- actuator: List<Actuator>

+ checkCapacity(): Boolean
+ issuePermit(): LoadPermit
+ confirmExit(): void

0..*

LoadPermit
- vehicleId: Integer
- measuredWeight: Double
- used: Boolean

+ reserveCapacity(): void

reads

0..*

0..*

<<interface>> Sensor

+ sense(): void

0..*

<<interface>> Actuator

+ actuate(): void

commands

VehicleWeightSensor
- measuredWeight: Double

+ sense(): void
+ readWeight(): Double

BridgeOccupancySensor
- vehicleCount: Integer

+ sense(): void
+ confirmExit(): Boolean

BridgeEntryGate
- closed: Boolean

+ actuate(): void
+ grantEntry(): void

AdmissionDisplay
- permitId: String

+ actuate(): void
+ showDecision(): void

Reused class signatures follow the original image set. New controllers, records and concrete devices extend that core.

Solid line: association | Filled diamond: composition | Dashed line + hollow triangle: interface realization

The new scenarioController association is additive. Repeated primitive fields express the same types as the originals.

35 Setup

35.1 Scenario

Electric vehicles compete for charging capacity after the station's available power is reduced.

35.2 Entities

1. Electric vehicles waiting for or receiving charge.
2. Charging station scheduler.
3. Charging units and station power meter.
4. Station operator and electricity capacity service.

35.3 Type of Communication

Vehicle-to-Charging Infrastructure Communication; Charging Infrastructure-to-Capacity Service Communication.

35.4 Description

A charging station receives a reduced power allowance while several vehicles have charging reservations. There are enough physical charging bays, but insufficient electrical capacity to meet every requested charging rate. Vehicles have different battery levels, connector requirements and declared departure times. The scheduler must revise charging allocations and explain why a vehicle receives a lower rate or a later start.

35.5 Dialogues

1. Is there a conflict?
Yes. The vehicles request charging allocations whose combined power exceeds the station's current allowance.

2. Is the reason good?
The requests reflect travel needs, but declared urgency must be assessed using the station's published allocation policy and available evidence.

3. Is settlement good?
Yes, if the station offers feasible charging slots or reduced rates and users can accept an alternative. Unverified departure claims must not silently override the queue policy.

4. Dialogue type
Negotiation: the station and vehicles seek an allocation under a shared power constraint. Information seeking establishes compatibility and the energy actually requested.

35 Setup (continued)

35.6 Solutions

1. Confirm the current power allowance, charger availability and connector compatibility.
2. Collect each vehicle's requested energy, reported battery level and departure preference, marking unverified information explicitly.
3. Compute a schedule that stays within the power allowance and follows the configured fairness and reservation policy.
4. Offer revised slots or rates, explain the constraints affecting each offer and record acceptance or cancellation.
5. Recalculate when a session ends or the allowance changes; recommend another reachable station only when range and availability are supported by current information.

35.7 PEAS

1. **Performance Measure** - Total allocated power stays within the allowance; requested energy delivered; waiting time; allocation fairness under the stated policy.
2. **Environment** - A charging station with multiple bays, compatible charging units, reserved sessions and a changing electrical power allowance.
3. **Actuators** - Charger power controllers, reservation update service, queue display and vehicle notification transmitter.
4. **Sensors** - Station power meter, charger connection sensors, charging session meter and receiver for vehicle battery reports.

35.8 Cascading Problems

1. A reserved vehicle does not arrive, leaving a slot unused until its reservation expires.
2. A charger disconnects unexpectedly and its remaining energy request must be rescheduled.
3. Several stations simultaneously redirect vehicles to one station, creating another capacity shortage.

Scenario 35 - Reviewed Class Diagram

AutonomousSystem

- trafficSignals: List<TrafficSignal>
- vehicles: List<Vehicle>
- roadNetwork: RoadNetwork
- scenarioController: ChargingScheduler

+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignal(trafficSignal: TrafficSignal): void
+ addVehicle(vehicle: Vehicle): void
+ removeVehicle(vehicle: Vehicle): void
+ setRoadNetwork(roadNetwork: RoadNetwork): void
+ runSimulation(): void

RoadNetwork

- intersections: List<Intersection>
- roads: List<Road>

+ addIntersection(intersection: Intersection): void
+ removeIntersection(intersection: Intersection): void
+ addRoad(road: Road): void
+ removeRoad(road: Road): void
+ getIntersections(): List<Intersection>
+ getRoads(): List<Road>

Vehicle

- id: Integer
- topLeft: Point
- bottomRight: Point
- destination: Point
- currentSpeed: Double
- maxSpeed: Double
- sensor: List<Sensor>
- actuator: List<Actuator>

+ move(): void
+ accelerate(): void
+ decelerate(): void
+ changeDirection(): void
+ setDestination(destination: Point): void

TrafficSignal

- id: Integer
- location: Point
- status: SignalStatus
- sensor: List<Sensor>
- actuator: List<Actuator>

+ getId(): Integer
+ getLocation(): Point
+ setStatus(status: SignalStatus): void
+ getStatus(): SignalStatus

0..*

0..*

0..*

0..*

Road

- id: Integer
- start: Intersection
- end: Intersection
- lanes: Integer
- capacity: Integer

+ getId(): Integer
+ getStart(): Intersection
+ getEnd(): Intersection
+ getLanes(): Integer
+ getCapacity(): Integer

start

end

Intersection

- id: Integer
- location: Point
- trafficSignals: List<TrafficSignal>

+ getId(): Integer
+ getLocation(): Point
+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignals(trafficSignal: TrafficSignal): void
+ getTrafficSignals(): List<TrafficSignal>

uses

Point

- x: Integer
- y: Integer

+ getX(): Integer
+ getY(): Integer
+ setX(Integer): void
+ setY(Integer): void

SignalStatus (enum)

RED
YELLOW
GREEN

coordinates vehicles

uses Road

scenario extension

ChargingScheduler

- powerAllowance: Double
- allocatedPower: Double
- sessions: List<ChargingSession>
- road: Road
- sensor: List<Sensor>
- actuator: List<Actuator>

+ checkCompatibility(): Boolean
+ allocatePower(): void
+ reviseSchedule(): void

0..*

ChargingSession

- requestedEnergy: Double
- departurePreference: Time
- accepted: Boolean

+ confirmOffer(): void

reads

0..*

0..*

<<interface>> Sensor

+ sense(): void

0..*

<<interface>> Actuator

+ actuate(): void

commands

StationPowerMeter

- availablePower: Double

+ sense(): void
+ readPower(): Double

ChargingSessionMeter

- deliveredEnergy: Double

+ sense(): void
+ readEnergy(): Double

ChargerPowerController

- allocatedPower: Double

+ actuate(): void
+ setPowerLimit(): void

ChargingQueueDisplay

- nextSessionId: String

+ actuate(): void
+ showOffer(): void

Reused class signatures follow the original image set. New controllers, records and concrete devices extend that core.

Solid line: association | Filled diamond: composition | Dashed line + hollow triangle: interface realization

The new scenarioController association is additive. Repeated primitive fields express the same types as the originals.

Time denotes a timestamp or declared departure time.

36 Setup

36.1 Scenario

A connected truck platoon must split because its members need different highway exits.

36.2 Entities

1. Platoon leader and following trucks.
2. Platoon coordination system.
3. Nearby vehicles occupying adjacent lanes.
4. Roadside communication unit.

36.3 Type of Communication

Inter Vehicle Communication; Vehicle-to-Roadside Infrastructure Communication.

36.4 Description

A connected truck platoon approaches a highway exit. A truck in the middle needs that exit, while the leader and other followers will continue. The platoon's normal following arrangement does not provide a confirmed gap for the truck to leave. Its membership, spacing and leadership assignments must be changed without assuming that nearby traffic will make room.

36.5 Dialogues

1. Is there a conflict?
Yes. One member needs to leave the platoon, while the other members are following the current formation and nearby vehicles already occupy the adjacent lane.

2. Is the reason good?
Yes. Members can have different destinations, but a planned exit does not justify an unconfirmed lane change.

3. Is settlement good?
Yes, when affected members acknowledge the split and a suitable gap is confirmed. If no gap becomes available, the truck continues to an alternative exit and receives an updated route.

4. Dialogue type
Deliberation: the members agree on a sequence for separating the formation. Negotiation may be needed for a voluntary gap request to nearby traffic.

36 Setup (continued)

36.6 Solutions

1. The departing truck sends its exit intention and the coordination system identifies the affected members.
2. Propose a split plan that identifies temporary leaders, spacing changes and the order of actions; collect acknowledgements.
3. Increase the relevant following gaps within the vehicles' operating constraints and monitor adjacent traffic.
4. Authorize the planned departure only after the required gap and member states are confirmed; cancel the maneuver if conditions change.
5. Update platoon membership after separation. If the exit is missed, preserve stable movement and request another route.

36.7 PEAS

1. Performance Measure - Separation completed without conflicting maneuvers; acknowledgement completeness; stable spacing; successful membership updates.

2. Environment - A connected truck platoon travelling near a highway exit with surrounding traffic and limited opportunities to change lanes.

3. Actuators - Vehicle acceleration and braking systems, turn indicators and platoon message transmitters.

4. Sensors - Inter-vehicle distance sensors, adjacent-lane detectors, vehicle position reports and message acknowledgement receiver.

36.8 Cascading Problems

1. A follower misses the split message and continues using the old leader assignment.
2. An unrelated vehicle enters the intended gap before separation is completed.
3. Two members request different exits at nearly the same time, requiring a revised split sequence.

Scenario 36 - Reviewed Class Diagram

AutonomousSystem

- trafficSignals: List<TrafficSignal>
- vehicles: List<Vehicle>
- roadNetwork: RoadNetwork
- scenarioController: PlatoonCoordinator

+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignal(trafficSignal: TrafficSignal): void
+ addVehicle(vehicle: Vehicle): void
+ removeVehicle(vehicle: Vehicle): void
+ setRoadNetwork(roadNetwork: RoadNetwork): void
+ runSimulation(): void

RoadNetwork

- intersections: List<Intersection>
- roads: List<Road>

+ addIntersection(intersection: Intersection): void
+ removeIntersection(intersection: Intersection): void
+ addRoad(road: Road): void
+ removeRoad(road: Road): void
+ getIntersections(): List<Intersection>
+ getRoads(): List<Road>

Vehicle

- id: Integer
- topLeft: Point
- bottomRight: Point
- destination: Point
- currentSpeed: Double
- maxSpeed: Double
- sensor: List<Sensor>
- actuator: List<Actuator>

+ move(): void
+ accelerate(): void
+ decelerate(): void
+ changeDirection(): void
+ setDestination(destination: Point): void

TrafficSignal

- id: Integer
- location: Point
- status: SignalStatus
- sensor: List<Sensor>
- actuator: List<Actuator>

+ getId(): Integer
+ getLocation(): Point
+ setStatus(status: SignalStatus): void
+ getStatus(): SignalStatus

0..*

0..*

0..*

0..*

Road

- id: Integer
- start: Intersection
- end: Intersection
- lanes: Integer
- capacity: Integer

+ getId(): Integer
+ getStart(): Intersection
+ getEnd(): Intersection
+ getLanes(): Integer
+ getCapacity(): Integer

start

end

Intersection

- id: Integer
- location: Point
- trafficSignals: List<TrafficSignal>

+ getId(): Integer
+ getLocation(): Point
+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignals(trafficSignal: TrafficSignal): void
+ getTrafficSignals(): List<TrafficSignal>

uses

Point

- x: Integer
- y: Integer

+ getX(): Integer
+ getY(): Integer
+ setX(Integer): void
+ setY(Integer): void

SignalStatus (enum)

RED
YELLOW
GREEN

coordinates vehicles

uses Road

scenario extension

PlatoonCoordinator

- leaderId: Integer
- memberIds: List<Integer>
- splitAcknowledged: Boolean
- road: Road
- sensor: List<Sensor>
- actuator: List<Actuator>

+ proposeSplit(): void
+ collectAcknowledgements(): void
+ updateMembership(): void

0..*

PlatoonSplitPlan

- departingVehicleId: Integer
- newLeaderId: Integer
- requiredGap: Double

+ validateGap(): Boolean

reads

<<interface>> Sensor

+ sense(): void

0..*

0..*

<<interface>> Actuator

+ actuate(): void

0..*

commands

InterVehicleGapSensor

- gap: Double

+ sense(): void
+ measureGap(): Double

AdjacentLaneSensor

- gapAvailable: Boolean

+ sense(): void
+ checkLane(): Boolean

SpacingController

- targetGap: Double

+ actuate(): void
+ adjustSpacing(): void

PlatoonMessageTransmitter

- planId: String

+ actuate(): void
+ broadcastSplit(): void

Reused class signatures follow the original image set. New controllers, records and concrete devices extend that core.

Solid line: association | Filled diamond: composition | Dashed line + hollow triangle: interface realization

The new scenarioController association is additive. Repeated primitive fields express the same types as the originals.

37 Setup

37.1 Scenario

A toll lane cannot confirm whether a payment succeeded after a communication timeout.

37.2 Entities

1. Vehicle at the toll lane.

2. Toll lane controller and payment tag reader.

3. Payment processing service.

4. Toll operator and vehicles waiting behind.

37.3 Type of Communication

Vehicle-to-Toll Infrastructure Communication; Toll Infrastructure-to-Payment Service Communication.

37.4 Description

A vehicle's toll payment request is sent, but the network connection times out before the lane receives the result. The vehicle may already have been charged, while the lane still displays an unpaid status. Retrying as a new transaction could create a duplicate charge. The controller must establish the original transaction's state and manage the waiting vehicle under the operator's exception policy.

37.5 Dialogues

1. Is there a conflict?
There is a disagreement about payment status: the vehicle reports a charge while the lane has no confirmed result. Intentional non-payment has not been established.

2. Is there a common problem to be solved?
Yes. Both parties need the original transaction's status so that passage can be handled without an unsupported second charge.

3. Is it theoretical?
No. A vehicle is waiting at the lane and the status must be resolved or handled through the configured exception procedure.

4. Dialogue type
Inquiry: the participants examine transaction evidence to establish what happened. Information seeking is used to request the result for the original transaction identifier.

37 Setup (continued)

37.6 Solutions

1. Keep the original transaction identifier and record the timeout as pending rather than automatically failed.
2. Query the payment service using that identifier; do not create a new payment request merely because the response was lost.
3. If success is confirmed, mark the transaction paid and allow passage according to the lane's vehicle-presence interlock.
4. If failure is confirmed, explain it and offer the permitted payment alternatives. If the status remains unknown, invoke the operator's exception procedure.
5. Where an exception bay is available, direct the vehicle there only when the movement is clear; reconcile the pending transaction later.

37.7 PEAS

1. Performance Measure - Duplicate charges avoided; correct payment-status reconciliation; time spent blocking the lane; proportion of unresolved transactions.

2. Environment - A toll plaza with connected payment processing, a controlled entry barrier, waiting vehicles and an optional exception bay.

3. Actuators - Toll barrier, lane status display, exception-bay guidance indicator and operator alert service.

4. Sensors - Payment tag reader, vehicle-presence detector, barrier position sensor and payment-status response receiver.

37.8 Cascading Problems

1. A delayed success response arrives after the vehicle has been moved to an exception bay.
2. A second read of the same tag is mistakenly treated as a new vehicle visit.
3. A network outage affects several lanes and requires coordinated exception handling.

Scenario 37 - Reviewed Class Diagram

AutonomousSystem

- trafficSignals: List<TrafficSignal>
- vehicles: List<Vehicle>
- roadNetwork: RoadNetwork
- scenarioController: TollLaneController

+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignal(trafficSignal: TrafficSignal): void
+ addVehicle(vehicle: Vehicle): void
+ removeVehicle(vehicle: Vehicle): void
+ setRoadNetwork(roadNetwork: RoadNetwork): void
+ runSimulation(): void

RoadNetwork

- intersections: List<Intersection>
- roads: List<Road>

+ addIntersection(intersection: Intersection): void
+ removeIntersection(intersection: Intersection): void
+ addRoad(road: Road): void
+ removeRoad(road: Road): void
+ getIntersections(): List<Intersection>
+ getRoads(): List<Road>

Vehicle

- id: Integer
- topLeft: Point
- bottomRight: Point
- destination: Point
- currentSpeed: Double
- maxSpeed: Double
- sensor: List<Sensor>
- actuator: List<Actuator>

+ move(): void
+ accelerate(): void
+ decelerate(): void
+ changeDirection(): void
+ setDestination(destination: Point): void

TrafficSignal

- id: Integer
- location: Point
- status: SignalStatus
- sensor: List<Sensor>
- actuator: List<Actuator>

+ getId(): Integer
+ getLocation(): Point
+ setStatus(status: SignalStatus): void
+ getStatus(): SignalStatus

0..*

0..*

0..*

0..*

Road

- id: Integer
- start: Intersection
- end: Intersection
- lanes: Integer
- capacity: Integer

+ getId(): Integer
+ getStart(): Intersection
+ getEnd(): Intersection
+ getLanes(): Integer
+ getCapacity(): Integer

start

end

Intersection

- id: Integer
- location: Point
- trafficSignals: List<TrafficSignal>

+ getId(): Integer
+ getLocation(): Point
+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignals(trafficSignal: TrafficSignal): void
+ getTrafficSignals(): List<TrafficSignal>

uses

Point

- x: Integer
- y: Integer

+ getX(): Integer
+ getY(): Integer
+ setX(Integer): void
+ setY(Integer): void

SignalStatus (enum)

RED
YELLOW
GREEN

coordinates vehicles

uses Road

scenario extension

TollLaneController

- pendingTransactionId: String
- paymentStatus: PaymentStatus
- exceptionRequired: Boolean
- road: Road
- sensor: List<Sensor>
- actuator: List<Actuator>

+ queryOriginalTransaction(): void
+ reconcileStatus(): void
+ applyExceptionPolicy(): void

0..*

TollTransaction

- transactionId: String
- vehicleId: Integer
- status: PaymentStatus

+ recordResult(): void

reads

<<interface>> Sensor

+ sense(): void

0..*

0..*

<<interface>> Actuator

+ actuate(): void

0..*

commands

PaymentTagReader

- tagId: String

+ sense(): void
+ readTag(): String

LanePresenceSensor

- vehiclePresent: Boolean

+ sense(): void
+ checkPresence(): Boolean

TollBarrier

- closed: Boolean

+ actuate(): void
+ allowPassage(): void

TollStatusDisplay

- statusText: String

+ actuate(): void
+ showPendingStatus(): void

Reused class signatures follow the original image set. New controllers, records and concrete devices extend that core.

Solid line: association | Filled diamond: composition | Dashed line + hollow triangle: interface realization

The new scenarioController association is additive. Repeated primitive fields express the same types as the originals.

PaymentStatus = PENDING, PAID, FAILED, UNKNOWN.

38 Setup

38.1 Scenario

A driver does not acknowledge a takeover request when automated driving reaches its operating boundary.

38.2 Entities

1. Vehicle operating in an automated driving mode.
2. Driver expected to take over control.
3. Driver monitoring and takeover management systems.
4. Roadside assistance coordinator and nearby road users.

38.3 Type of Communication

Vehicle-to-Driver Communication; Vehicle-to-Infrastructure Communication; Inter Vehicle Communication where available.

38.4 Description

An automated vehicle approaches the end of the conditions in which its current driving mode can operate. It issues a takeover request, but the driver gives no acknowledgement and the monitoring system cannot confirm readiness. The vehicle must distinguish a request being delivered from control actually being accepted and use its defined fallback procedure if takeover does not occur.

38.5 Dialogues

1. Is there a conflict?
No intentional conflict is established. The system requires a change of control, but the driver's ability to respond is unknown.

2. Is there a common problem to be solved?
Yes. The vehicle and the driver need a confirmed control state while the operating boundary approaches.

3. Is it theoretical?
No. The system must act within its remaining operating capability and cannot assume that a silent driver has accepted control.

4. Dialogue type
Information seeking followed by deliberation if the driver responds. If there is no response, a defined fallback state transition replaces further dialogue; consent is not invented.

38 Setup (continued)

38.6 Solutions

1. Detect the approaching operating boundary and issue the configured visual, audible and tactile takeover prompts.
2. Check both explicit acknowledgement and the available driver-readiness evidence; keep control ownership explicit.
3. Transfer control only when the required readiness and acceptance conditions are satisfied.
4. If those conditions are not met, execute the vehicle's defined minimal-risk fallback within its current capability, using a suitable refuge only if one is verified reachable.
5. Communicate the vehicle's condition to nearby traffic where supported and request assistance when the fallback procedure calls for it.

38.7 PEAS

1. Performance Measure - No unacknowledged control transfer; timely fallback initiation; correctly reported control ownership; assistance request completion.

2. Environment - An automated vehicle nearing an operating boundary with an unresponsive or unavailable driver and surrounding traffic.

3. Actuators - Takeover display, audible and tactile alerts, braking and steering controls, hazard indicators and assistance transmitter.

4. Sensors - Driver monitoring sensor, takeover acknowledgement input, vehicle capability monitor and surrounding-object sensors.

38.8 Cascading Problems

1. The driver responds after fallback has begun, requiring an explicit reassessment before any control transfer.
2. The preferred refuge becomes unavailable and another fallback option must be assessed.
3. A monitoring fault produces uncertain readiness information and must be reported as uncertainty.

Scenario 38 - Reviewed Class Diagram

AutonomousSystem

- trafficSignals: List<TrafficSignal>
- vehicles: List<Vehicle>
- roadNetwork: RoadNetwork
- scenarioController: TakeoverManager

+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignal(trafficSignal: TrafficSignal): void
+ addVehicle(vehicle: Vehicle): void
+ removeVehicle(vehicle: Vehicle): void
+ setRoadNetwork(roadNetwork: RoadNetwork): void
+ runSimulation(): void

RoadNetwork

- intersections: List<Intersection>
- roads: List<Road>

+ addIntersection(intersection: Intersection): void
+ removeIntersection(intersection: Intersection): void
+ addRoad(road: Road): void
+ removeRoad(road: Road): void
+ getIntersections(): List<Intersection>
+ getRoads(): List<Road>

Vehicle

- id: Integer
- topLeft: Point
- bottomRight: Point
- destination: Point
- currentSpeed: Double
- maxSpeed: Double
- sensor: List<Sensor>
- actuator: List<Actuator>

+ move(): void
+ accelerate(): void
+ decelerate(): void
+ changeDirection(): void
+ setDestination(destination: Point): void

TrafficSignal

- id: Integer
- location: Point
- status: SignalStatus
- sensor: List<Sensor>
- actuator: List<Actuator>

+ getId(): Integer
+ getLocation(): Point
+ setStatus(status: SignalStatus): void
+ getStatus(): SignalStatus

0..*

0..*

0..*

0..*

Road

- id: Integer
- start: Intersection
- end: Intersection
- lanes: Integer
- capacity: Integer

+ getId(): Integer
+ getStart(): Intersection
+ getEnd(): Intersection
+ getLanes(): Integer
+ getCapacity(): Integer

start

end

Intersection

- id: Integer
- location: Point
- trafficSignals: List<TrafficSignal>

+ getId(): Integer
+ getLocation(): Point
+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignals(trafficSignal: TrafficSignal): void
+ getTrafficSignals(): List<TrafficSignal>

uses

Point

- x: Integer
- y: Integer

+ getX(): Integer
+ getY(): Integer
+ setX(Integer): void
+ setY(Integer): void

SignalStatus (enum)

RED
YELLOW
GREEN

coordinates vehicles

uses Road

scenario extension

TakeoverManager

- controlOwner: ControlOwner
- driverReady: Boolean
- boundaryReached: Boolean
- road: Road
- sensor: List<Sensor>
- actuator: List<Actuator>

+ requestTakeover(): void
+ confirmAcceptance(): Boolean
+ startFallback(): void

0..*

TakeoverRequest

- requestId: String
- acknowledged: Boolean
- readinessVerified: Boolean

+ mayTransferControl(): Boolean

reads

<<interface>> Sensor

+ sense(): void

0..*

0..*

<<interface>> Actuator

+ actuate(): void

0..*

commands

DriverReadinessSensor

- readinessKnown: Boolean

+ sense(): void
+ assessReadiness(): Boolean

OperatingBoundaryMonitor

- remainingCapability: String

+ sense(): void
+ checkBoundary(): Boolean

TakeoverAlert

- alertLevel: Integer

+ actuate(): void
+ requestDriverResponse(): void

FallbackMotionController

- fallbackState: String

+ actuate(): void
+ executeFallback(): void

Reused class signatures follow the original image set. New controllers, records and concrete devices extend that core.

Solid line: association | Filled diamond: composition | Dashed line + hollow triangle: interface realization

The new scenarioController association is additive. Repeated primitive fields express the same types as the originals.

ControlOwner = AUTOMATION, DRIVER, FALLBACK.

39 Setup

39.1 Scenario

Vehicle positioning sources disagree about which of two parallel roads the vehicle occupies.

39.2 Entities

1. Connected vehicle and its navigation system.

2. Roadside positioning units.

3. Localization consistency service.

4. Route guidance service.

39.3 Type of Communication

Vehicle-to-Roadside Infrastructure Communication; Inter Infrastructure Communication.

39.4 Description

A vehicle travels where an elevated road and a lower service road run close together. Its satellite-based position is matched to one road, but roadside observations indicate the other. The two roads have different permitted movements. The guidance system must resolve the location uncertainty before issuing a road-specific maneuver or claiming that the vehicle missed a turn.

39.5 Dialogues

1. Is there a conflict?
There is conflicting evidence about location, rather than a known conflict between road users.

2. Is there a common problem to be solved?
Yes. The vehicle and guidance service need a sufficiently supported road assignment before selecting a maneuver.

3. Is it theoretical?
No. An incorrect assignment could produce an inappropriate instruction while the vehicle continues moving.

4. Dialogue type
Inquiry: the participants compare independent observations and their reliability. The outcome can remain unresolved if the evidence does not support one road.

39 Setup (continued)

39.6 Solutions

1. Detect the disagreement and retain both candidate road assignments with their evidence and timestamps.
2. Request additional observations such as roadside beacon identity, elevation evidence, odometry or a recognized road feature.
3. Compare the evidence without treating several copies of one source as independent confirmation.
4. While the road assignment is unresolved, withhold the uncertain road-specific maneuver and state the positioning limitation; the driving system continues according to its own operating procedure.
5. Once the assignment is supported, recompute guidance and explain which observations resolved the discrepancy.

39.7 PEAS

1. Performance Measure - Correct road assignment on reviewed examples; time to resolve disagreement; unsupported maneuver instructions avoided; uncertainty reported accurately.

2. Environment - Nearby parallel or vertically separated roads with imperfect satellite reception and intermittent roadside positioning coverage.

3. Actuators - Navigation display, route guidance update service and uncertainty notification transmitter.

4. Sensors - Satellite position receiver, odometry sensor, roadside beacon receiver and available elevation or road-feature observations.

39.8 Cascading Problems

1. An outdated map disagrees with otherwise consistent position observations.
2. A roadside beacon is associated with the wrong road in the infrastructure database.
3. Position estimates oscillate between roads and require a stable decision before guidance resumes.

Scenario 39 - Reviewed Class Diagram

AutonomousSystem

- trafficSignals: List<TrafficSignal>
- vehicles: List<Vehicle>
- roadNetwork: RoadNetwork
- scenarioController: LocalizationResolver

+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignal(trafficSignal: TrafficSignal): void
+ addVehicle(vehicle: Vehicle): void
+ removeVehicle(vehicle: Vehicle): void
+ setRoadNetwork(roadNetwork: RoadNetwork): void
+ runSimulation(): void

RoadNetwork

- intersections: List<Intersection>
- roads: List<Road>

+ addIntersection(intersection: Intersection): void
+ removeIntersection(intersection: Intersection): void
+ addRoad(road: Road): void
+ removeRoad(road: Road): void
+ getIntersections(): List<Intersection>
+ getRoads(): List<Road>

Vehicle

- id: Integer
- topLeft: Point
- bottomRight: Point
- destination: Point
- currentSpeed: Double
- maxSpeed: Double
- sensor: List<Sensor>
- actuator: List<Actuator>

+ move(): void
+ accelerate(): void
+ decelerate(): void
+ changeDirection(): void
+ setDestination(destination: Point): void

TrafficSignal

- id: Integer
- location: Point
- status: SignalStatus
- sensor: List<Sensor>
- actuator: List<Actuator>

+ getId(): Integer
+ getLocation(): Point
+ setStatus(status: SignalStatus): void
+ getStatus(): SignalStatus

0..*

0..*

0..*

0..*

Road

- id: Integer
- start: Intersection
- end: Intersection
- lanes: Integer
- capacity: Integer

+ getId(): Integer
+ getStart(): Intersection
+ getEnd(): Intersection
+ getLanes(): Integer
+ getCapacity(): Integer

start

end

Intersection

- id: Integer
- location: Point
- trafficSignals: List<TrafficSignal>

+ getId(): Integer
+ getLocation(): Point
+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignals(trafficSignal: TrafficSignal): void
+ getTrafficSignals(): List<TrafficSignal>

uses

Point

- x: Integer
- y: Integer

+ getX(): Integer
+ getY(): Integer
+ setX(Integer): void
+ setY(Integer): void

SignalStatus (enum)

RED
YELLOW
GREEN

coordinates vehicles

uses Road

scenario extension

LocalizationResolver

- candidateRoadIds: List<Integer>
- resolvedRoadId: Integer
- locationConfirmed: Boolean
- road: Road
- sensor: List<Sensor>
- actuator: List<Actuator>

+ compareObservations(): void
+ requestMoreEvidence(): void
+ resolveRoad(): Boolean

0..*

PositionObservation

- sourceId: String
- roadId: Integer
- timestamp: Time

+ isCurrent(): Boolean

reads

0..*

0..*

<<interface>> Sensor

+ sense(): void

0..*

<<interface>> Actuator

+ actuate(): void

commands

SatellitePositionSensor

- estimatedRoadId: Integer

+ sense(): void
+ readPosition(): Point

RoadsideBeaconReceiver

- beaconRoadId: Integer

+ sense(): void
+ readBeacon(): String

NavigationGuidanceDisplay

- locationUncertain: Boolean

+ actuate(): void
+ showUncertainty(): void

RouteGuidanceUpdater

- confirmedRoadId: Integer

+ actuate(): void
+ recomputeGuidance(): void

Reused class signatures follow the original image set. New controllers, records and concrete devices extend that core.

Solid line: association | Filled diamond: composition | Dashed line + hollow triangle: interface realization

The new scenarioController association is additive. Repeated primitive fields express the same types as the originals.

Time denotes an observation timestamp.

40 Setup

40.1 Scenario

Connected vehicles receive contradictory incident messages whose sources or timestamps cannot all be verified.

40.2 Entities

1. Vehicles receiving road incident messages.
2. Roadside communication units.
3. Message verification service.
4. Traffic management operator.

40.3 Type of Communication

Inter Vehicle Communication; Vehicle-to-Infrastructure Communication; Inter Infrastructure Communication.

40.4 Description

Vehicles receive one message stating that a road is closed and another stating that it is open. One report may be old, relayed repeatedly or associated with an unverifiable source. The system has not established whether any sender is malicious. It must assess authenticity, freshness and independent supporting observations before presenting a route recommendation as confirmed.

40.5 Dialogues

1. Is there a conflict?
There is a conflict between claims about the road's state. A deliberate dispute or attack has not been proven.

2. Is there a common problem to be solved?
Yes. Receivers need a supported incident state and a clear account of which messages remain uncertain.

3. Is it theoretical?
No. Vehicles may change routes immediately in response to the reports, spreading the effect of an incorrect message.

4. Dialogue type
Inquiry: the system compares claims, source verification, timestamps and corroborating observations. Repetition alone is not treated as proof.

40 Setup (continued)

40.6 Solutions

1. Collect the messages with their identifiers, source information, timestamps and claimed road state.
2. Check message integrity and source verification using the configured trust mechanism; flag failed checks without inventing a cause.
3. Check freshness and detect repeated or relayed copies so one original claim is not counted as several independent reports.
4. Seek a current infrastructure observation or operator confirmation when the claims still disagree.
5. Display confirmed information separately from unresolved claims. Issue a revised advisory when the evidence changes and retain the decision basis for review.

40.7 PEAS

1. Performance Measure - Incorrect reports prevented from becoming confirmed advisories; time to resolve contradictory claims; duplicate reports recognized; evidence traceability.

2. Environment - A connected road network where vehicles and roadside units exchange incident messages with different ages and verification states.

3. Actuators - Incident advisory display, verified-message broadcaster, operator alert service and guidance update interface.

4. Sensors - Message receiver, infrastructure observation feed, timestamp reference and source-verification inputs.

40.8 Cascading Problems

1. A valid message arrives late and is stale even though its source is authentic.
2. Several vehicles relay the same original report, creating a false appearance of independent agreement.
3. The authoritative source is unavailable and the system must preserve an unresolved status.

Scenario 40 - Reviewed Class Diagram

AutonomousSystem

- trafficSignals: List<TrafficSignal>
- vehicles: List<Vehicle>
- roadNetwork: RoadNetwork
- scenarioController: IncidentVerificationService

+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignal(trafficSignal: TrafficSignal): void
+ addVehicle(vehicle: Vehicle): void
+ removeVehicle(vehicle: Vehicle): void
+ setRoadNetwork(roadNetwork: RoadNetwork): void
+ runSimulation(): void

RoadNetwork

- intersections: List<Intersection>
- roads: List<Road>

+ addIntersection(intersection: Intersection): void
+ removeIntersection(intersection: Intersection): void
+ addRoad(road: Road): void
+ removeRoad(road: Road): void
+ getIntersections(): List<Intersection>
+ getRoads(): List<Road>

Vehicle

- id: Integer
- topLeft: Point
- bottomRight: Point
- destination: Point
- currentSpeed: Double
- maxSpeed: Double
- sensor: List<Sensor>
- actuator: List<Actuator>

+ move(): void
+ accelerate(): void
+ decelerate(): void
+ changeDirection(): void
+ setDestination(destination: Point): void

TrafficSignal

- id: Integer
- location: Point
- status: SignalStatus
- sensor: List<Sensor>
- actuator: List<Actuator>

+ getId(): Integer
+ getLocation(): Point
+ setStatus(status: SignalStatus): void
+ getStatus(): SignalStatus

0..*

0..*

0..*

0..*

Road

- id: Integer
- start: Intersection
- end: Intersection
- lanes: Integer
- capacity: Integer

+ getId(): Integer
+ getStart(): Intersection
+ getEnd(): Intersection
+ getLanes(): Integer
+ getCapacity(): Integer

start

end

Intersection

- id: Integer
- location: Point
- trafficSignals: List<TrafficSignal>

+ getId(): Integer
+ getLocation(): Point
+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignals(trafficSignal: TrafficSignal): void
+ getTrafficSignals(): List<TrafficSignal>

uses

Point

- x: Integer
- y: Integer

+ getX(): Integer
+ getY(): Integer
+ setX(Integer): void
+ setY(Integer): void

SignalStatus (enum)

RED
YELLOW
GREEN

coordinates vehicles

uses Road

scenario extension

IncidentVerificationService

- reports: List<IncidentReport>
- confirmedState: String
- statusUnresolved: Boolean
- road: Road
- sensor: List<Sensor>
- actuator: List<Actuator>

+ verifySources(): void
+ removeDuplicateClaims(): void
+ resolveIncidentState(): void

0..*

IncidentReport

- sourceId: String
- timestamp: Time
- claimedState: String

+ checkFreshness(): Boolean

reads

0..*

0..*

<<interface>> Sensor

+ sense(): void

0..*

<<interface>> Actuator

+ actuate(): void

commands

IncidentMessageReceiver

- messageId: String

+ sense(): void
+ receiveReport(): IncidentReport

RoadsideObservationSensor

- observedState: String

+ sense(): void
+ readObservation(): String

VerifiedAdvisoryDisplay

- confirmedState: String

+ actuate(): void
+ showEvidenceStatus(): void

IncidentUpdateTransmitter

- advisoryId: String

+ actuate(): void
+ broadcastRevision(): void

Reused class signatures follow the original image set. New controllers, records and concrete devices extend that core.

Solid line: association | Filled diamond: composition | Dashed line + hollow triangle: interface realization

The new scenarioController association is additive. Repeated primitive fields express the same types as the originals.

Time denotes an incident-report timestamp.

41 Setup

41.1 Scenario

A bus receives a departure request while its door system cannot confirm that the doorway is clear.

41.2 Entities

1. Bus and driver.
2. Boarding or alighting passengers.
3. Door controller and passenger detection system.
4. Bus departure interlock.

41.3 Type of Communication

Passenger-to-Vehicle Communication; Inter-System Communication within the Bus; Driver-to-Vehicle Communication.

41.4 Description

A bus is ready to leave a stop, but a doorway detector reports a passenger or object in the closing area. The door position sensor gives an inconsistent closed indication. The driver requests departure to keep the service on schedule. The bus must reconcile the doorway evidence and keep movement permission dependent on a verified door state.

41.5 Dialogues

1. Is there a conflict?
Yes. The departure request conflicts with unresolved evidence that the doorway may still be occupied.

2. Is the reason good?
Keeping the timetable is a valid service goal, but it does not establish that the door is secure or the doorway clear.

3. Is settlement good?
Yes, if departure is delayed until the obstruction is resolved and closure is verified. A continuing sensor disagreement requires inspection under the operator's procedure.

4. Dialogue type
Information seeking establishes the door and passenger state, followed by deliberation about clearing the doorway or requesting assistance.

41 Setup (continued)

41.6 Solutions

1. Keep departure permission disabled when the door state or doorway clearance is unresolved.
2. Notify the driver and passengers of the reported obstruction or sensor disagreement without claiming a passenger is at fault.
3. Use the door's defined obstruction-response procedure and request the doorway to be cleared.
4. Recheck obstruction detection and secure-closure indication before enabling departure; reset permission if either condition changes.
5. If evidence remains inconsistent, request inspection or assistance and communicate the resulting service delay.

41.7 PEAS

1. Performance Measure - No departure with an unverified door state; obstruction detection and resolution time; false clearance decisions; passenger communication clarity.

2. Environment - A bus stopped for passenger exchange with a powered door, a departure interlock and possibly conflicting sensor reports.

3. Actuators - Door drive, departure interlock, driver warning display and passenger announcement system.

4. Sensors - Doorway obstruction detector, door-latch sensor, door position sensor and driver departure-request input.

41.8 Cascading Problems

1. A strap or small object remains trapped despite the passenger having moved away.
2. A faulty latch sensor repeatedly prevents a verified departure and requires maintenance.
3. A following bus is delayed by the occupied stop and needs separate service coordination.

Scenario 41 - Reviewed Class Diagram

AutonomousSystem

- trafficSignals: List<TrafficSignal>
- vehicles: List<Vehicle>
- roadNetwork: RoadNetwork
- scenarioController: BusDepartureController

+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignal(trafficSignal: TrafficSignal): void
+ addVehicle(vehicle: Vehicle): void
+ removeVehicle(vehicle: Vehicle): void
+ setRoadNetwork(roadNetwork: RoadNetwork): void
+ runSimulation(): void

RoadNetwork

- intersections: List<Intersection>
- roads: List<Road>

+ addIntersection(intersection: Intersection): void
+ removeIntersection(intersection: Intersection): void
+ addRoad(road: Road): void
+ removeRoad(road: Road): void
+ getIntersections(): List<Intersection>
+ getRoads(): List<Road>

Vehicle

- id: Integer
- topLeft: Point
- bottomRight: Point
- destination: Point
- currentSpeed: Double
- maxSpeed: Double
- sensor: List<Sensor>
- actuator: List<Actuator>

+ move(): void
+ accelerate(): void
+ decelerate(): void
+ changeDirection(): void
+ setDestination(destination: Point): void

TrafficSignal

- id: Integer
- location: Point
- status: SignalStatus
- sensor: List<Sensor>
- actuator: List<Actuator>

+ getId(): Integer
+ getLocation(): Point
+ setStatus(status: SignalStatus): void
+ getStatus(): SignalStatus

0..*

0..*

0..*

0..*

Road

- id: Integer
- start: Intersection
- end: Intersection
- lanes: Integer
- capacity: Integer

+ getId(): Integer
+ getStart(): Intersection
+ getEnd(): Intersection
+ getLanes(): Integer
+ getCapacity(): Integer

start

end

Intersection

- id: Integer
- location: Point
- trafficSignals: List<TrafficSignal>

+ getId(): Integer
+ getLocation(): Point
+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignals(trafficSignal: TrafficSignal): void
+ getTrafficSignals(): List<TrafficSignal>

uses

Point

- x: Integer
- y: Integer

+ getX(): Integer
+ getY(): Integer
+ setX(Integer): void
+ setY(Integer): void

SignalStatus (enum)

RED
YELLOW
GREEN

coordinates vehicles

uses Road

scenario extension

BusDepartureController

- doorwayClear: Boolean
- latchVerified: Boolean
- departureEnabled: Boolean
- road: Road
- sensor: List<Sensor>
- actuator: List<Actuator>

+ assessDoorState(): void
+ requestClearance(): void
+ authorizeDeparture(): Boolean

0..*

DoorStateReport

- obstructionDetected: Boolean
- latchClosed: Boolean
- reportsAgree: Boolean

+ isVerifiedClear(): Boolean

reads

0..*

0..*

<<interface>> Sensor

+ sense(): void

0..*

<<interface>> Actuator

+ actuate(): void

commands

DoorwayObstructionSensor

- obstructed: Boolean

+ sense(): void
+ detectObstruction(): Boolean

DoorLatchSensor

- latched: Boolean

+ sense(): void
+ confirmLatch(): Boolean

BusDoorDrive

- doorPosition: String

+ actuate(): void
+ applyObstructionResponse(): void

DepartureInterlock

- enabled: Boolean

+ actuate(): void
+ setDeparturePermission(): void

Reused class signatures follow the original image set. New controllers, records and concrete devices extend that core.

Solid line: association | Filled diamond: composition | Dashed line + hollow triangle: interface realization

The new scenarioController association is additive. Repeated primitive fields express the same types as the originals.

42 Setup

42.1 Scenario

Tunnel air-quality measurements exceed a configured threshold while ventilation capacity is reduced.

42.2 Entities

1. Vehicles inside and approaching the tunnel.
2. Tunnel operations controller.
3. Air-quality monitoring stations and ventilation equipment.
4. Tunnel operator and entrance control units.

42.3 Type of Communication

Tunnel Infrastructure-to-Vehicle Communication; Inter Infrastructure Communication; Infrastructure-to-Operator Communication.

42.4 Description

Tunnel monitoring stations report air-quality measurements above an operator-configured action threshold. A ventilation status report also indicates that part of the ventilation system is unavailable. The tunnel remains physically unobstructed. The operations controller must check the measurements and coordinate ventilation, admission control and operator response under the tunnel's defined incident procedure.

42.5 Dialogues

1. Is there a conflict?
No intentional conflict is established. Vehicles and the operator share a problem involving the tunnel's environmental condition.

2. Is there a common problem to be solved?
Yes. The controller must respond to the reported condition while considering vehicles already inside and those approaching the entrances.

3. Is it theoretical?
No. Current measurements and reduced ventilation capacity require an operational response, even before the cause is fully known.

4. Dialogue type
Deliberation: the systems and operator choose a coordinated response. Inquiry is used to assess sensor agreement and equipment status.

42 Setup (continued)

42.6 Solutions

1. Compare current readings from the monitoring stations and check timestamps and sensor health; preserve disagreement as uncertainty.
2. Confirm available ventilation capacity and execute the configured response for the reported threshold condition.
3. Apply the incident procedure's admission restrictions and notify approaching vehicles of the tunnel's current status.
4. Provide vehicles already inside with the operator-approved instructions for the verified incident state; escalate when measurements or equipment status require it.
5. Restore normal admission only after the procedure's recovery conditions are met and the operator or authorized controller records clearance.

42.7 PEAS

1. Performance Measure - Time from a verified threshold condition to the configured response; reduction in threshold exceedance duration; consistent entrance status; premature reopening avoided.

2. Environment - An occupied road tunnel with air-quality stations, controllable entrances and partially unavailable ventilation equipment.

3. Actuators - Ventilation fans and dampers, entrance control signals, variable message signs and operator alert transmitter.

4. Sensors - Air-quality sensors, airflow sensors, ventilation health monitors and tunnel occupancy detectors.

42.8 Cascading Problems

1. A sensor drifts and reports high values inconsistent with other stations.
2. Ventilation status messages stop arriving during the response, making available capacity uncertain.
3. Measurements briefly improve and then worsen, so reopening requires the specified recovery conditions rather than one favorable reading.

Scenario 42 - Reviewed Class Diagram

AutonomousSystem

- trafficSignals: List<TrafficSignal>
- vehicles: List<Vehicle>
- roadNetwork: RoadNetwork
- scenarioController: TunnelEnvironmentController

+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignal(trafficSignal: TrafficSignal): void
+ addVehicle(vehicle: Vehicle): void
+ removeVehicle(vehicle: Vehicle): void
+ setRoadNetwork(roadNetwork: RoadNetwork): void
+ runSimulation(): void

RoadNetwork

- intersections: List<Intersection>
- roads: List<Road>

+ addIntersection(intersection: Intersection): void
+ removeIntersection(intersection: Intersection): void
+ addRoad(road: Road): void
+ removeRoad(road: Road): void
+ getIntersections(): List<Intersection>
+ getRoads(): List<Road>

Vehicle

- id: Integer
- topLeft: Point
- bottomRight: Point
- destination: Point
- currentSpeed: Double
- maxSpeed: Double
- sensor: List<Sensor>
- actuator: List<Actuator>

+ move(): void
+ accelerate(): void
+ decelerate(): void
+ changeDirection(): void
+ setDestination(destination: Point): void

TrafficSignal

- id: Integer
- location: Point
- status: SignalStatus
- sensor: List<Sensor>
- actuator: List<Actuator>

+ getId(): Integer
+ getLocation(): Point
+ setStatus(status: SignalStatus): void
+ getStatus(): SignalStatus

0..*

0..*

0..*

0..*

Road

- id: Integer
- start: Intersection
- end: Intersection
- lanes: Integer
- capacity: Integer

+ getId(): Integer
+ getStart(): Intersection
+ getEnd(): Intersection
+ getLanes(): Integer
+ getCapacity(): Integer

start

end

Intersection

- id: Integer
- location: Point
- trafficSignals: List<TrafficSignal>

+ getId(): Integer
+ getLocation(): Point
+ addTrafficSignal(trafficSignal: TrafficSignal): void
+ removeTrafficSignals(trafficSignal: TrafficSignal): void
+ getTrafficSignals(): List<TrafficSignal>

uses

Point

- x: Integer
- y: Integer

+ getX(): Integer
+ getY(): Integer
+ setX(Integer): void
+ setY(Integer): void

SignalStatus (enum)

RED
YELLOW
GREEN

coordinates vehicles

uses Road

scenario extension

TunnelEnvironmentController

- actionThreshold: Double
- ventilationAvailable: Boolean
- recoveryVerified: Boolean
- road: Road
- sensor: List<Sensor>
- actuator: List<Actuator>

+ assessMeasurements(): void
+ applyIncidentProcedure(): void
+ authorizeReopening(): Boolean

0..*

TunnelConditionReport

- airQualityReading: Double
- sensorHealthy: Boolean
- timestamp: Time

+ exceedsThreshold(): Boolean

reads

0..*

0..*

<<interface>> Sensor

+ sense(): void

0..*

<<interface>> Actuator

+ actuate(): void

commands

AirQualitySensor

- reading: Double

+ sense(): void
+ measureAirQuality(): Double

VentilationHealthMonitor

- availableCapacity: Double

+ sense(): void
+ checkVentilation(): Boolean

VentilationController

- fanCommand: String

+ actuate(): void
+ applyResponse(): void

TunnelEntryControl

- admissionAllowed: Boolean

+ actuate(): void
+ setAdmissionState(): void

Reused class signatures follow the original image set. New controllers, records and concrete devices extend that core.

Solid line: association | Filled diamond: composition | Dashed line + hollow triangle: interface realization

The new scenarioController association is additive. Repeated primitive fields express the same types as the originals.

Time denotes a measurement timestamp.